

Day 3 Hands-On Labs

Docker Deep Dive & Kubernetes Intro — Practical Exercises

Lab 1: Dockerfile & Commands

Lab 2: Networking

Lab 3: Volumes

Lab 4: Docker Compose

Lab 1

Basic Docker Commands & Dockerfile Review

 **Objective:** Refresh your memory on the most common Docker commands and practice building a custom image using a Dockerfile.

1. The Dockerfile Keywords

Create a simple directory and a `Dockerfile` inside it:

```
mkdir docker-lab1 && cd docker-lab1
touch Dockerfile
```

Open the `Dockerfile` and add the following:

```
# Use a lightweight base image
FROM nginx:alpine

# Set the working directory
WORKDIR /usr/share/nginx/html

# Create a custom index.html using the RUN command
RUN echo "<h1>Hello from DevOps Lab 1</h1>" > index.html

# Expose port 80
EXPOSE 80

# The default command to run
CMD ["nginx", "-g", "daemon off;"]
```

2. Build the Image

Use the `docker build` command to create an image named `my-nginx-app`:

```
docker build -t my-nginx-app:v1 .
```

3. List Local Images

Validate that your image was created successfully:

```
docker images
```

4. Run the Container

Spin up a container from your newly created image:

```
docker run -d -p 8080:80 --name my-web-server my-nginx-app:v1
```

5. Check Running Containers

```
docker ps
```

Access `http://localhost:8080` in your browser to see your custom HTML!

6. Stop and Review All Containers

```
# Stop the container gracefully
docker stop my-web-server

# View ALL containers, including stopped ones
docker ps -a
```

7. Tag and Push (Simulated)

```
docker tag my-nginx-app:v1 yourusername/my-nginx-app:v1
# docker push yourusername/my-nginx-app:v1
```

✅ **Expected Outcome:** You have practically reviewed how a Dockerfile is structured (FROM, WORKDIR, RUN, EXPOSE, CMD) and navigated the complete container lifecycle using `build`, `run`, `ps`, and `stop`.

 **Objective:** Understand the practical difference between the default bridge network and custom bridge networks by pinging containers by name.

1. The Default Bridge Network (No DNS)

Run two interactive containers on the default network:

```
docker run -d --name container-a alpine sleep 3600
docker run -d --name container-b alpine sleep 3600
```

Try to ping `container-a` from inside `container-b`:

```
docker exec -it container-b ping container-a
```

 **Result:** `ping: bad address 'container-a'`. The default bridge network does NOT support automatic DNS resolution by container name.

2. Create a Custom Network

```
docker network create devops-net
```

3. Run Containers on the Custom Network

```
docker run -d --name web-app --network devops-net alpine sleep 3600
docker run -d --name db-app --network devops-net alpine sleep 3600
```

4. Test DNS Resolution

Ping `db-app` from `web-app`:

```
docker exec -it web-app ping -c 3 db-app
```

 **Result:** Success! The ping replies show the IP address automatically resolved by Docker's internal DNS server.

5. Cleanup

```
docker rm -f container-a container-b web-app db-app
docker network rm devops-net
```

 **Expected Outcome:** You validated that custom Docker networks provide built-in DNS resolution, allowing containers to communicate using their names instead of unpredictable IP addresses.

 **Objective:** Prove experimentally that containers are ephemeral (data is lost on deletion) and solve it by attaching a persistent Docker Volume.

Part 1: The Ephemeral Disaster

Step 1 — Create data inside a container

```
docker run -it --name test-box ubuntu:latest /bin/bash
```

Inside the container:

```
echo "CRITICAL FINANCIAL DATA" > /mydatabase.txt
cat /mydatabase.txt
exit
```

Step 2 — Delete the container and try again

```
docker rm test-box
docker run -it --name test-box ubuntu:latest cat /mydatabase.txt
```

 **Result:** `cat: /mydatabase.txt: No such file or directory`. The data is permanently lost!

Part 2: The Volume Solution

Step 1 — Create a managed Docker Volume

```
docker rm -f test-box
docker volume create my-persistent-data
```

Step 2 — Validate and inspect the volume

```
docker volume ls
docker volume inspect my-persistent-data
```

Step 3 — Run container with the volume mounted

```
docker run -it --name test-box -v my-persistent-data:/data ubuntu:latest /bin/bash
```

Inside the container:

```
echo "IMMORTAL DATA" > /data/file.txt
exit
```

Step 4 — Destroy the container

```
docker rm test-box
```

Part 3: Validate Persistence

Spin up a completely new container, attaching the same volume:

```
docker run -it --name new-box -v my-persistent-data:/data ubuntu:latest cat /data/file.txt
```

✅ **Result:** "IMMORTAL DATA". The file survived the container destruction!

✅ **Expected Outcome:** You practically experienced the transient nature of container file systems and successfully mitigated it using Docker managed volumes, validating the data persistence across container lifecycles.

 **Objective:** Experience how Docker Compose solves the multi-container orchestration problem by spinning up WordPress + MySQL using a single declarative YAML file.

Features Covered

- Multi-container dependencies (`depends_on`)
- Automatic network creation and service linking
- Volume provisioning via Compose
- `docker-compose` specific commands

1. Create the Directory Context

```
mkdir compose-lab && cd compose-lab
```

2. Create the Compose File

Create a file exactly named `docker-compose.yml`:

```
version: '3.8'

services:
  db:
    image: mysql:8.0
    volumes:
      - db_data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppassword

  wordpress:
    image: wordpress:latest
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppassword
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - db

volumes:
  db_data:
```

3. Start the Environment

```
docker-compose up -d
```

 **Note:** Docker Compose will automatically download the images, create a custom bridge network called `compose-lab_default`, provision the `db_data` volume, and start the DB first (because of `depends_on`).

4. Validate the Environment

Check the Running Services:

```
docker-compose ps
```

Check the Auto-Created Network:

```
docker network ls | grep compose-lab
```

Check the Auto-Created Volume:

```
docker volume ls | grep db_data
```

View Combined Logs:

```
docker-compose logs -f
```

(Press Ctrl+C to exit the logs)

5. Access the Web Server

Open a browser and navigate to **http://localhost:8080**. You should see the WordPress installation screen successfully communicating with the MySQL database!

6. Cleanup

```
docker-compose down
```

 **Note:** The volume `db_data` is NOT removed by default. To wipe everything including persistent data, run `docker-compose down -v`.

 **Expected Outcome:** You successfully orchestrated two connected containers using declarative YAML and validated the resources (networks, volumes) automatically provisioned by Docker Compose.